SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Analytical Problem Solving
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	ASWIN · M	**Reg. No.:**	192412437
Section / Batch:	B·E (ECE)	**Date:**	05/08/2026

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A computer system uses 8-bit 2's complement representation to process signed integer arithmetic. Consider two numbers: −52 and +19. Analyze in detail how the processor performs both addition and subtraction, including binary conversion, sign extension, intermediate steps, and the final result. Determine whether overflow occurs in each operation and explain how the processor detects and handles overflow.
2. A modern processor is designed with an 8-bit carry look-ahead adder to accelerate arithmetic operations. Given two binary operands, analyze how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time of both adders. Justify why CLA is preferred in high-performance processors.
3. A digital processor performs signed multiplication using Booth's algorithm for efficient arithmetic computation. Multiply +11 and −7, and analyze each iteration of the algorithm by showing the contents of the accumulator, multiplier register, Booth bit, and the arithmetic right shifts. Explain how Booth's algorithm minimizes the number of addition and subtraction operations while correctly handling signed operands.
4. A binary arithmetic unit supports both restoring and non-restoring division techniques for integer division. Analyze the execution of the division of 25 ÷ 4 using both methods by showing the intermediate remainder values, quotient generation, and register contents after each step. Compare the algorithms based on execution time, hardware complexity, and computational efficiency, and explain why one method is preferred over the other.
5. A floating-point unit (FPU) processes real numbers according to the IEEE 754 standard. Analyze the representation of the decimal number +26.75 in both single-precision (32-bit) and double-precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, analyze the floating-point addition of 26.75 and −8.5, explaining exponent alignment, mantissa operations, normalization, rounding, and the effect of precision on the final result.

Code of Conduct

I ASWIN · M – 192412437 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Aswin · M

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator **Faculty Incharge**

01/08/2026

Aswin. M
192412437.
CSA1220
Computer Architecture.

CO2 ASSESSMENT TOOL - 1.

(1) Given.

A = −52
B = +19

$52_{10} = 00110100_2$

−52 :

00110100 ⟶ 11001011 ⟶ 11001100

+19 = 00010011

a) Addition

−52 + 19

  11001100
+ 00010011
  11011111

  00100000
         1
  00100001 = 33

11011111_2 = −33

−52 + 19 = −33

b) Subtraction:

-52 - 19

-19

```
  00010011
  --------
  11101100
         1
  --------
  11101101  = -19
  --------

  11001100
  11101101
  ---------
 110111001
 ---------
```

```
  10111001
  01000110
  --------
         1
  --------
  01000111  = 71
  --------

  --------
  10111000  = -71
  --------
```

-52 - 19 = -71

---

(2) 8-bit carry look-Ahead Adder.

A = 10110101

B = 01101110

$G_i = A_i B_i$

$P_i = A_i \oplus B_i$

Tabulation:

Bit	A_i	B_i	G_i
0	1	0	0
1	0	1	0
2	1	1	1
3	0	1	0
4	1	0	0
5	1	1	1
6	0	1	0
7	1	0	0

$C_0 = 0$

$C_1 = G_0 + P_0 C_0 = 0$

$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 0$

$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

$C_3 = 1$

$C_4 = G_3 + P_3 C_3 = 0 + 1(1) = 1$

$C_5 = G_4 + P_4 C_4 = 0 + 1(1) = 1$

$C_6 = G_5 + P_5 C_5 = 1 + 0(1) = 1$

$C_7 = G_6 + P_6 C_6 = 0 + 1(1) = 1$

$C_8 = G_7 + P_7 C_7 = 0 + 1(1) = 1$

Sum

$S_i = P_i \oplus C_i$

$S = 00100011$

$C_8 = 1$.

$$
\begin{array}{r}
10110101 \\
01101110 \\
\hline
1\ 00100011 \\
\hline
\end{array}
$$

$10110101_2 + 01101110_2 = 100100011_2$.

<u>execution time comparision</u>

$T_{ripple\ carry} \approx 8T_{carry}$

$T_{CLA} = T_{generate} + T_{carry-look\ ahead} + T_{sum}$.

$T_{CLA} < T_{RCA}$

CLA is faster because carries are calculated in parallel .

(3) <u>Booth's Algorithm</u>

$11 \times (-7)$

5 - bit registers

$+11 = 01011$

$-11 = 10101$

$-7 = 11001$

A = 00000

Q = 11001

$Q_{-1} = 0$

Iteration 1

$Q_0 Q_{-1} = 10$

A = A − M

00000 − 01011 = 10101

Arithmatic right shift.

A	Q	Q-1
10101	11001	0
11010	11100	1

Iteration 2.

$Q_0 Q_{-1} = 01$

A = A + M

11010 + 1011 = 100001

5-bit result.

A = 00001

Arithmatic right shift.

A	Q	Q-1
00001	11100	1
00000	11110	0

Iteration 3

$Q_0 Q_{-1} = 01$

$A = A + M$ no operation

Arithmatic right shift

A	Q	Q-1
00000	11110	0
00000	01111	0

Iteration 4

$Q_0 Q_{-1} = 10$

$A = A - M$

00000 - 01011 = 10101

Arithmatic right shift

A	Q	Q-1
10101	01111	0
11010	10111	1

Iteration 5

$Q_0 Q_{-1} = 11$

No operation

Arithmatic right shift

A	Q	Q-1
11010	10111	1
11101	01011	1

Final product

AQ = 11101 01011

$-77 = 111011\,0011_2$

$11 \times (-7) = -77$

$25 \div 4$

$25 = 11001_2$

$4 = 00100_2$.

use 5-bit registers

A = 00000

Q = 11001

M = 00100

A) Restoring Division

A = 00000

Q = 11001

M = 00100

00000 11001 → 00001 10010

00001 − 00100 = −3

Restore :

A = 00001 $Q_0 = 0$.

Q = 10010

00001 10010 → 00011 00100

00011 − 00100 = −1.

$Q_0 = 0$

00011 00100 → 00100 01000

00100 − 00100 = 00010

$Q_0 = 1$

00010 01001 → 00100 10010

00100 − 00100 = 00000

$Q_0 = 1$.

00000 10011 → 00001 00110

00001 − 00100 = −3

$Q_0 = 0$

$Q = 00110_2$

$= 6$

$R = 00001_2 = 1$

$25 \div 4$

$= 6$

remainder

$= 1$

B) <u>Non - Restoring division</u>

A = 00000

Q = 11001

M = 00100

00000 11001 → 00001 10010

A = −3

Q = 10010

$Q_0 = 0$.

−3 → −5

A < 0

A + M = −5 + 4 = −1

$-1 \rightarrow -2$

$A < 0$

$-2 + 4 = 2$

$Q_0 = 1$

$0 \rightarrow 1$

$A \geq 0$

$A = -3$

$Q = 00110$

$Q_0 = 0$

$Q = (00110)_2 = 6$

$R = (00001)_2 = 1$

$25 \div 4 = 6$ remainder 1.

---

(5) IEEE 754 Floating - point calculations

a) $+26.75$

in Binary

$26.75 = (11010.11)_2$

normalize it

$11010.11 = 1.101011 \times 2^4$.

Single precision - 32 - bit

$S = 0$

$E = 4 + 127 = 131$

$131 = (10000011)_2$.

Double precision - 64 bit

$S = 0$

$E = 4 + 1023 = 1027.$

$1027_{10} = 10000000011_2.$

b) $26.75 + (-8.5)$

$8.5 = 1000.1_2.$

$= 1.0001 \times 2^3$

$-8.5 = -1.0001 \times 2^3$

$26.75 = 1.101011 \times 2^4$

$-8.5 = -1.0001 \times 2^3$

-1.0001×2^3

$= -0.10001 \times 2^4.$

1.101011

$-0.100010.$

$$\begin{array}{r} 1.101011 \\ -\ 0.100010 \\ \hline 1.001001 \\ \hline \end{array}$$

$1.001001 \times 2^4.$

$1.001001_2 \times 16$

$= 18.25$

$26.75 + (-8.5)$

$= 18.25$

Final result $= 18.25$